

Table of Contents

Abstract

1. Introduction
2. Background
3. Aims and objective
4. Dataset description
5. Data preprocessing
 - 5.1 Data imputation using KNN technique
 - 5.2 Data imputation using MICE technique
 - 5.3 Removing null values
6. Exploratory Data Analysis
 - 6.1 Statistical summary
 - 6.2 Visualizations
7. Data preparation
 - 7.1 Features and target label
 - 7.2 Data normalization
 - 7.3 Data splitting into train and test
8. Implementation of algorithms and model evaluation
 - 8.1 Random Forest
 - 8.1.1 Summary of approach
 - 8.1.2 Applying Random Forest
 - 8.1.3 Model evaluation
 - 8.2 Support Vector Machine
 - 8.2.1 Summary of approach
 - 8.2.2 Applying Support Vector Machine
 - 8.2.3 Model evaluation
 - 8.3 XGBoosting
 - 8.3.1 Summary of approach
 - 8.3.2 Applying XGBoosting
 - 8.3.3 Model evaluation
9. Results and comparison
10. Conclusion ,Recommendations and Future work
 - 10.1 Conclusion
 - 10.2 Recommendations
 - 10.3 Future work

References

Abstract

This report investigates the development of a predictive model for vertical farming cube door status, employing artificial intelligence and machine learning techniques. The objective is to automate door operations based on environmental conditions, enhancing plant growth efficiency while minimizing energy consumption. The dataset encompasses environmental parameters such as temperature, humidity, and door status. Three machine learning algorithms—Random Forest, Support Vector Machine (SVM), and XGBoost—are evaluated for their predictive accuracy. The findings highlight the potential for optimizing environmental control and energy efficiency in vertical farming setups.

1. Introduction

Vertical farming represents an innovative approach to agriculture, leveraging technology to cultivate crops in vertically stacked layers as shown in [Figure 1](#). In this context, the control of environmental parameters within farming cubes is critical for ensuring optimal growing conditions. One key aspect is the management of door status, which directly impacts internal temperature, humidity, and overall environmental stability.

The ability to predict door status in advance offers significant advantages, allowing for automated adjustments to maintain ideal growing conditions. By harnessing artificial intelligence and machine learning, we aim to develop a predictive model capable of accurately forecasting whether the door of a vertical farming cube will be open or closed.

This report outlines our methodology for building and evaluating predictive models using a dataset comprising environmental data from vertical farming cubes. We focus on three prominent machine learning algorithms—Random Forest, SVM, and XGBoost—and compare their performance in predicting door status. Through this analysis, we aim to demonstrate the potential of predictive modeling to optimize vertical farming operations, improving both crop yields and energy efficiency.

[Figure 1](#) : Image of verticle farming



2. Background

Vertical farming has emerged as a promising solution to address challenges in traditional agriculture, including limited arable land, water scarcity, and climate variability. This innovative approach involves growing crops in vertically stacked layers, often within controlled indoor environments. By utilizing advanced technologies such as hydroponics, aeroponics, and artificial lighting, vertical farms can efficiently produce high yields of fresh produce year-round.

One of the key advantages of vertical farming is its ability to tightly regulate environmental conditions to optimize plant growth. Parameters such as temperature, humidity, light intensity, and carbon dioxide levels can be precisely controlled within farming cubes, creating ideal growing conditions for various crops. Additionally, vertical farming offers benefits such as reduced water usage, minimal pesticide use, and decreased transportation distances, leading to more sustainable and resilient food production systems.

However, maintaining optimal environmental conditions within vertical farming setups requires continuous monitoring and management. In particular, the status of the farming cube doors plays a crucial role in regulating internal conditions. Opening or closing the doors can impact temperature, humidity, and airflow, affecting plant health and overall productivity. Automating the operation of farming cube doors based on predictive models offers significant advantages in terms of efficiency and resource utilization. By accurately forecasting door status using machine learning algorithms, vertical farming systems can adapt to changing environmental conditions in real-time, ensuring optimal growing conditions while minimizing energy consumption.

This project aims to leverage artificial intelligence and machine learning techniques to develop a predictive model for door status in vertical farming cubes. By analyzing environmental data collected from farming cubes, we seek to enhance operational efficiency and productivity in vertical farming systems, contributing to sustainable agriculture practices and food security.

3. Aims and objective

The aim of this project is to develop a robust predictive model utilizing advanced artificial intelligence and machine learning algorithms to accurately forecast the status of doors within vertical farming cubes.

The objectives of this project are:

- Gather comprehensive environmental data from vertical farming cubes, including temperature, humidity, and door status, ensuring data integrity and completeness.
- Implement rigorous data preprocessing techniques to handle missing values, outliers, and inconsistencies, facilitating reliable model training.
- Identify key environmental parameters influencing door status prediction, leveraging domain knowledge and exploratory data analysis.
- Perform feature engineering to extract relevant features and enhance the predictive capability of the model.
- Implement state-of-the-art machine learning algorithms, including Random Forest, Support Vector Machine (SVM), and XGBoost, tailored to the characteristics of the vertical farming dataset.
- Compare the performance metrics of different machine learning algorithms, such as accuracy, precision, recall, and F1-score, to identify the most effective approach for door status prediction.

4. Dataset description

The dataset encompasses records from 400,000 instances of advanced vertical farming cubes, serving as automated environments for seed cultivation. These cubes orchestrate vital agricultural processes such as watering, ventilation, and lighting control, fostering optimal conditions for plant growth. Each cube is uniquely identified by a Cube ID and is integrated into a network via Wi-Fi, transmitting data to a central backend system for continuous monitoring and management. Timestamps accompany each entry, detailing the date and time of data collection. Within the dataset, measurements of temperature and humidity are provided for two distinct layers (Layer A and Layer B) of the cube, denoted as Temperature Layer A, Temperature Layer B, Humidity Layer A, and Humidity Layer B, respectively. Moreover, a critical feature, Door status, indicates whether the door of the vertical farming cube is open or closed, influencing the internal environment's regulation. This dataset, originating from Kaggle in 2021 and presented in CSV format, offers a comprehensive view of environmental dynamics and operational statuses within vertical farming systems as shown in [Figure 2](#) . Analyzing this dataset enables the development of predictive models aimed at optimizing operational efficiency and enhancing crop yield in modern agricultural practices.

Figure 2 : Code for loading dataset

```
import pandas as pd

# Path to the CSV file
csv_file_path = "/content/verticle_farming_dataset.csv"

# Read the CSV file into a pandas DataFrame
df = pd.read_csv(csv_file_path)
df.head()
```

	Cube ID	Timestamp	Temperature Layer A	Temperature Layer B	Humidity Layer A	Humidity Layer B	Door
0	1924	2000-01-01 00:00:00	25.002968	19.698718	46.554858	42.477965	0
1	1894	2000-01-01 01:00:00	26.149738	23.073634	78.012732	81.200566	1
2	1011	2000-01-01 02:00:00	20.523260	18.463343	39.411010	25.432781	0
3	1111	2000-01-01 03:00:00	17.617682	19.175223	47.941998	24.875994	0
4	1024	2000-01-01 04:00:00	18.080046	16.520452	44.077954	44.203032	0

5. Data preprocessing

In preprocessing of the dataset, several steps were undertaken to address missing values and ensure data integrity. Initially, the Timestamp column was converted into a time format to facilitate temporal analysis. Subsequently, missing values were addressed using two distinct imputation techniques: K-Nearest Neighbors (KNN) and Multiple Imputation by Chained Equations (MICE) as shown in [Figure 3](#) and [Figure 4](#)

5.1 Data imputation using KNN technique

In the KNN imputation process, missing values in the Temperature Layer A, Temperature Layer B, Door, Humidity Layer A, and Humidity Layer B columns were estimated based on the values of their nearest neighbors. This method leverages the similarity between data points to infer missing values, thus preserving the underlying data structure.

Figure 3 : Code for data imputation using KNN technique

```
from sklearn.impute import KNNImputer
# Preprocessing: Convert non-numeric values to NaN
df = df.apply(pd.to_numeric, errors='coerce')

# KNN Imputation
def knn_impute(df):
    imputer = KNNImputer(n_neighbors=2) # You can adjust n_neighbors as needed
    df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)
    return df_imputed

# Impute missing values
df_imputed = knn_impute(df)

print("Original DataFrame:")
print(df)
print("\nDataFrame after KNN imputation:")
print(df_imputed)
```

5.2 Data imputation using MICE technique

Following KNN imputation, the MICE technique was employed to further refine the dataset. MICE iteratively imputes missing values by modeling each variable with missing data as a function of other variables, thereby capturing complex relationships within the dataset.

Figure 4 : Code for data imputation using MICE technique

```
import pandas as pd
from fancyimpute import IterativeImputer

# MICE Imputation
def mice_impute(df):
    imputer = IterativeImputer()
    df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)
    return df_imputed

# Impute missing values using MICE
df_imputed = mice_impute(df)

# Truncate the DataFrame to retain 32k rows
df_imputed = df_imputed.iloc[:32000]

# Save the DataFrame as 'vertical_farming_dataset.csv'
df_imputed.to_csv('vertical_farming_dataset.csv', index=False)

print("DataFrame after MICE imputation and truncation:")
print(df_imputed)
```

5.3 Removing null values

After completing the imputation steps, instances with remaining missing values were dropped from the dataset to ensure data completeness as shown in [Figure 5](#). As a result of these preprocessing steps, the dataset was significantly refined, resulting in a reduced size of 32,000 rows. This processed dataset now offers a robust foundation for subsequent analysis and modeling, with minimized data inconsistencies and enhanced reliability.

Figure 5 : Code for dropping null values

```
import pandas as pd
from fancyimpute import IterativeImputer

# Remove rows with missing values
df_cleaned = df.dropna()
```

6. Exploratory Data Analysis

6.1 Statistical summary

Upon conducting exploratory data analysis (EDA), it was determined that the dataset contains no missing values across all attributes after data preprocessing , ensuring data completeness. Data types were consistent throughout, with Cube ID and Timestamp represented as integers (int64), while environmental parameters such as Temperature and Humidity were denoted as floating-point numbers (float64). Additionally, summary statistics provided insights into the distribution and characteristics of the dataset, revealing average values and measures of variability for each attribute as shown in [Figure 6](#) . These findings underscore the dataset's reliability and suitability for subsequent analysis and modeling tasks.

Figure 6 : Statistical summary of the dataset

	Cube ID	Timestamp	Temperature Layer A	Temperature Layer B	Humidity Layer A	Humidity Layer B	Door
count	32000.000000	3.200000e+04	32000.000000	32000.000000	32000.000000	32000.000000	32000.000000
mean	1500.433937	1.004283e+09	22.474143	22.478858	54.885921	54.912765	0.495156
std	288.915283	3.325590e+07	3.901571	3.890907	18.093332	18.044769	0.499984
min	1000.000000	9.466848e+08	7.471589	8.633310	0.048306	3.086743	0.000000
25%	1249.000000	9.754839e+08	19.681300	19.710677	39.842329	39.791456	0.000000
50%	1501.000000	1.004283e+09	22.469422	22.471791	54.508376	54.831514	0.000000
75%	1750.000000	1.033082e+09	25.255063	25.267714	70.080251	69.926141	1.000000
max	1999.000000	1.061881e+09	37.680076	36.058152	109.217772	107.513759	1.000000

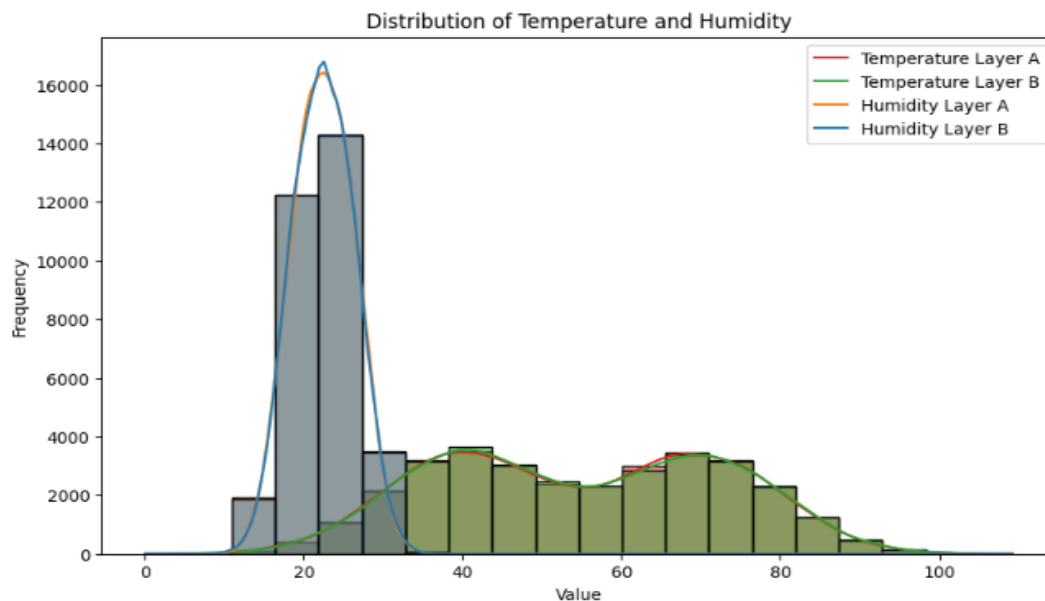
Missing values:

Missing values:			Data types:		
	Column	Missing Values		Column	Data Type
0	Cube ID	0	0	Cube ID	int64
1	Timestamp	0	1	Timestamp	int64
2	Temperature Layer A	0	2	Temperature Layer A	float64
3	Temperature Layer B	0	3	Temperature Layer B	float64
4	Humidity Layer A	0	4	Humidity Layer A	float64
5	Humidity Layer B	0	5	Humidity Layer B	float64
6	Door	0	6	Door	int64

6.1 Visualizations

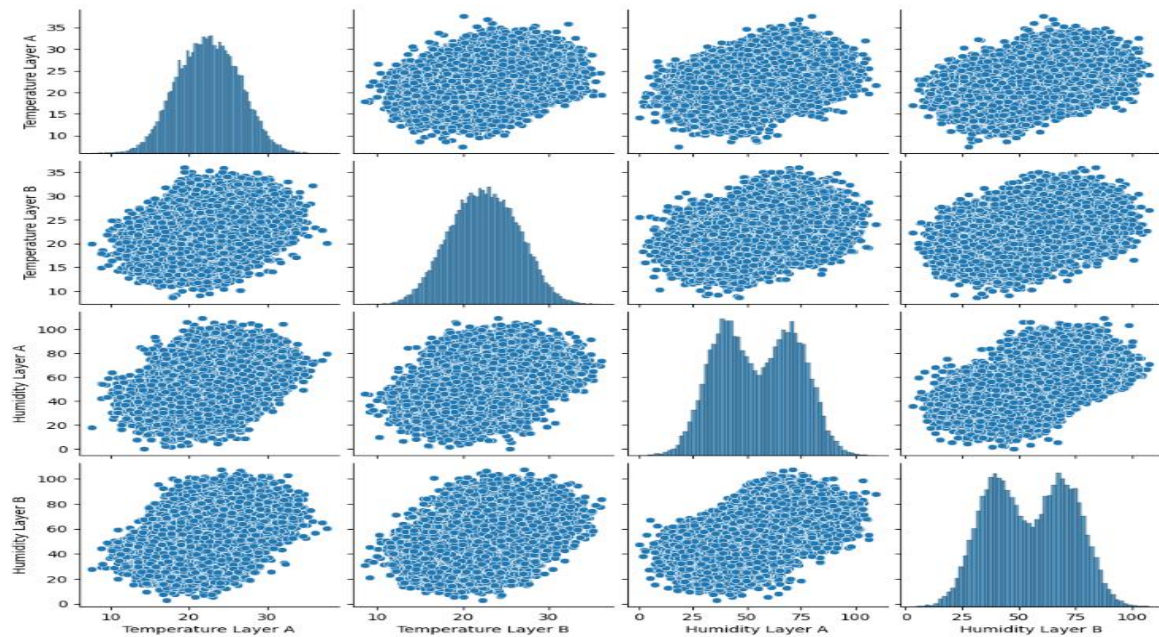
The visualization in [Figure 7](#) illustrates a striking dichotomy between the two layers: while Humidity Layer A exhibits lower humidity levels, Temperature Layer A coincides with notably elevated temperatures. In contrast, Humidity Layer B experiences higher humidity levels alongside comparatively moderated temperatures. This juxtaposition suggests a complex interplay between humidity and temperature, where drier conditions in Layer A are associated with heightened temperatures, while Layer B's higher humidity levels contribute to a more temperate climate. Such insights not only underscore the nuanced environmental dynamics within distinct layers but also hold implications for diverse fields, from ecological studies to urban planning, emphasizing the importance of understanding and adapting to spatial variations in climatic conditions.

Figure 7 : Graph for striking dichotomy between the two layers of Temperature and Humidity



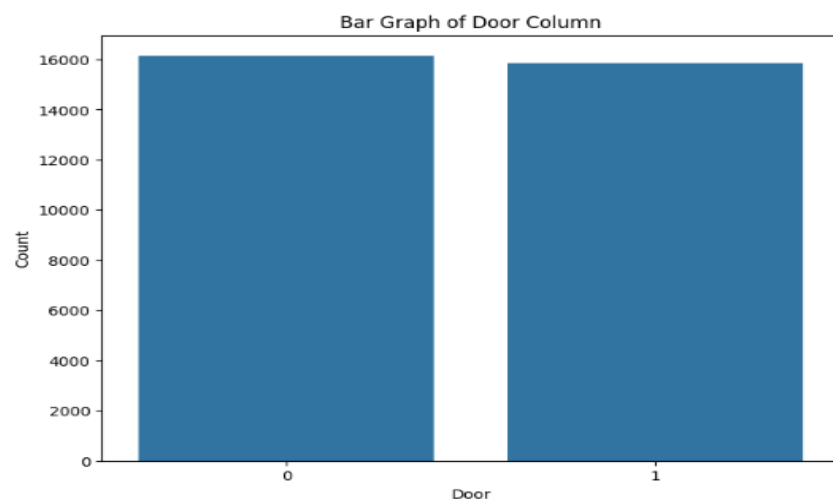
In [Figure 8](#) the pairplot visualization, a notable observation is the presence of scatterplots where temperature variables, such as 'Temperature Layer A' and 'Temperature Layer B', exhibit a clear positive correlation. This is evident from the linear trend displayed in the scatterplot, where an increase in temperature in one layer corresponds to a corresponding increase in the other layer. Additionally, scatterplots for humidity variables, particularly 'Humidity Layer A' and 'Humidity Layer B', also demonstrate a positive correlation, albeit with some variability. These findings suggest a coherent relationship between temperature and humidity across both layers, indicating that as temperatures rise or fall in one layer, humidity levels tend to follow a similar trend. Such strong correlations between temperature and humidity variables underscore the interconnectedness of these climatic factors within the studied environment, highlighting the potential for predictive modeling and further investigation into the underlying mechanisms driving these relationships.

Figure 8 : Pairplot visualizations



The dataset exhibits remarkable balance, as depicted by the bar plot showcasing an almost symmetrical distribution of instances across the "open" and "closed" states in the Door status variable as shown in [Figure 9](#). Each category is represented by a comparable number of observations, indicating a well-balanced dataset where both classes receive equal representation. This balanced distribution ensures that predictive models trained on this data are not skewed towards any particular outcome, fostering robust and unbiased predictions. Consequently, the bar plot serves as visual evidence of the dataset's inherent equilibrium, affirming its suitability for subsequent analyses and modeling tasks.

Figure 9 : Bar graph showing balanced dataset



7. Data preparation

In the process of data preparation for subsequent modeling endeavors, careful consideration was given to both feature selection and normalization techniques.

7.1 Features and target label

The chosen features, including 'Cube ID', 'Temperature Layer A', 'Temperature Layer B', 'Humidity Layer A', and 'Humidity Layer B', were identified as pertinent contributors to predicting the door status within vertical farming cubes. These features collectively encapsulate essential environmental parameters critical for discerning variations in door operations.

Figure 10 : Assigning Features and target label

```
#Features(X)
X = df[['Cube ID', 'Temperature Layer A', 'Temperature Layer B', 'Humidity Layer A', 'Humidity Layer B']]

# Target label (Y)
Y = df['Door']
```

7.2 Data normalization

To ensure uniformity and equitable treatment of features, the MinMaxScaler function from the sklearn.preprocessing module was applied. This scaling procedure normalized the feature values, aligning them within a common scale and facilitating equitable comparisons across variables. The resultant normalized feature set (X_normalized) thus provided a standardized basis for subsequent modeling tasks as shown in [Figure 11](#).

Figure 11 : Code for data normalization

```
from sklearn.preprocessing import MinMaxScaler

# Initialize the MinMaxScaler
scaler = MinMaxScaler()

# Fit and transform the features in X
X_normalized = scaler.fit_transform(X)

# Display the shape of X_normalized
print("Shape of X_normalized (normalized features):", X_normalized.shape)

Shape of X_normalized (normalized features): (32000, 5)
```

7.3 Data splitting into train and test

Subsequently, the dataset underwent stratified splitting into distinct training and testing subsets, a crucial step to preserve the integrity of the data for model evaluation. Leveraging the train_test_split function from the sklearn.model_selection module, an 80-20 partitioning scheme was employed, dedicating 80% of the data for model training and reserving the remaining 20% for independent validation purposes. Upon completion of the

partitioning process, meticulous scrutiny of the dimensions of both training and testing sets ensued. The training set (X_train and Y_train) encompassed 25,600 instances, whereas the testing set (X_test and Y_test) comprised 6,400 instances as shown in [Figure 12](#). These sets are now poised for the deployment of predictive models, equipped to discern and forecast door status in vertical farming cubes with a high degree of accuracy, thereby contributing to operational efficiency and optimization within such agricultural systems.

Figure 12 : Code for the splitting of dataset

```
from sklearn.model_selection import train_test_split

# Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, Y_train, Y_test = train_test_split(X_normalized, Y, test_size=0.2, random_state=42)

# Display the shapes of the training and testing sets
print("Shape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of Y_train:", Y_train.shape)
print("Shape of Y_test:", Y_test.shape)
```



```
Shape of X_train: (25600, 5)
Shape of X_test: (6400, 5)
Shape of Y_train: (25600,)
Shape of Y_test: (6400,)
```

8. Implementation of algorithms and model evaluation

Employing algorithms such as SVM, XGBoosting and random forests, we aimed to develop robust models capable of accurately predicting the door status in vertical farming cubes based on environmental parameters. Model evaluation was conducted using established metrics such as accuracy, precision, recall, and F1-score, providing comprehensive insights into the models' performance. Additionally, we strived to optimize model performance and enhance their applicability in real-world scenarios within vertical farming environments.

8.1 Random Forest

8.1.1 Summary of approach

Random Forest, a powerful ensemble learning method, was chosen for its robustness and ability to handle complex datasets with high-dimensional feature spaces. It operates by constructing multiple decision trees during training and outputs the mode of the classes (classification) or the mean prediction (regression) of the individual trees. By combining the predictions of multiple trees, Random Forest mitigates overfitting and enhances generalization performance, making it well-suited for our dataset with multiple environmental features and the binary classification task of predicting door status in vertical farming cubes.

8.1.2 Applying Random Forest

For the implementation of Random Forest on our dataset, we utilized the RandomForestClassifier from the sklearn.ensemble module. Following initialization, the classifier was trained on the training data (X_train and Y_train), leveraging the inherent parallelism and randomness of Random Forest to build an ensemble of decision trees. Subsequently, predictions were generated for the testing set (X_test), and model performance was assessed using the accuracy metric from sklearn.metrics. The calculated accuracy provides a quantitative measure of the classifier's effectiveness in accurately predicting the door status in vertical farming cubes based on the provided environmental features. This implementation showcases the practical application of Random Forest in agricultural contexts, offering insights into optimizing operational efficiency and ensuring optimal conditions for plant growth in vertical farming systems.

Figure 13 : Code for Random Forest algorithm

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Initialize the Random Forest classifier
rf_classifier = RandomForestClassifier()

# Train the Random Forest classifier on the training data
rf_classifier.fit(X_train, Y_train)

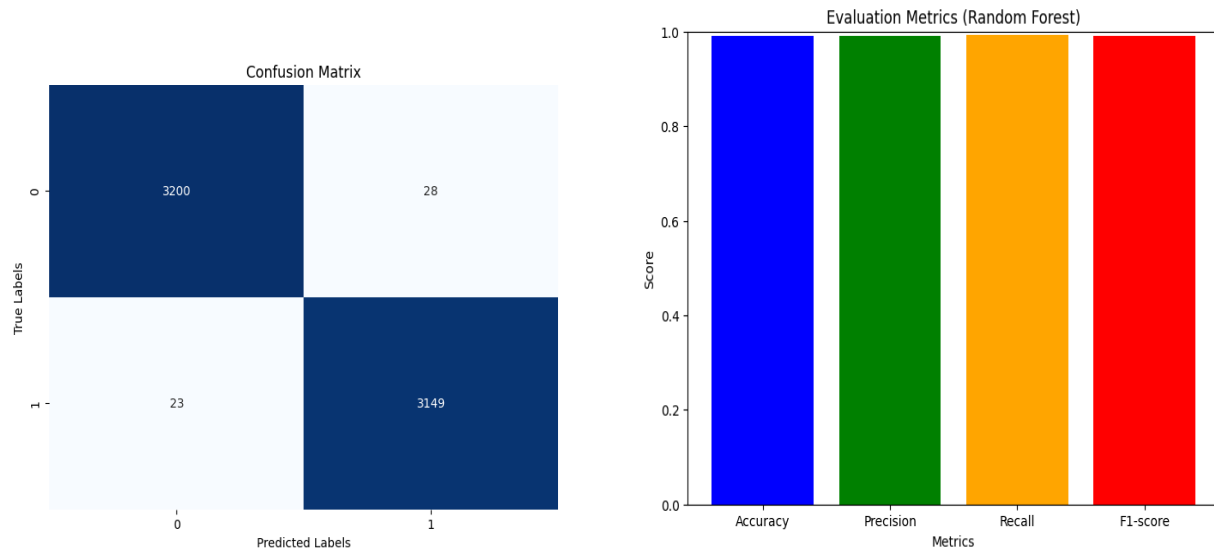
# Predictions on the testing set
Y_pred = rf_classifier.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(Y_test, Y_pred)
```

8.1.2 Model evaluation

Model evaluation provides critical insights into the performance of predictive algorithms and their applicability in real-world scenarios as shown in [Figure 14](#). In assessing the Random Forest classifier applied to our dataset, multiple metrics were considered. With an impressive accuracy score of 99.20%, the classifier demonstrates a high level of precision in correctly predicting the door status in vertical farming cubes. Precision, which measures the proportion of correctly predicted positive instances out of all instances predicted as positive, yielded a value of 99.12%, further underscoring the model's reliability in minimizing false positives. Similarly, the recall metric, indicating the proportion of correctly predicted positive instances out of all actual positive instances, achieved a commendable score of 99.27%. This high recall rate signifies the model's effectiveness in capturing true positive cases. The F1-score, a harmonic mean of precision and recall, stood at 99.20%, reflecting a balanced performance between precision and recall. Collectively, these evaluation metrics attest to the Random Forest classifier's robustness and accuracy in predicting door status based on environmental parameters, underscoring its potential to optimize operational efficiency and promote sustainable practices within vertical farming environments.

Figure 14 : Model evaluation graphs



8.2 Support Vector Machine

8.2.1 Summary of approach

Support Vector Machine (SVM) was selected for its capability to effectively handle both linear and non-linear classification tasks by identifying optimal hyperplanes in high-dimensional feature spaces. Its ability to accommodate complex datasets and its robustness against overfitting made SVM an ideal choice for our predictive modeling task.

8.2.2 Applying Support Vector Machine

For the implementation of SVM on our dataset, we utilized the SVC (Support Vector Classifier) module from the `sklearn.svm` library. Following initialization, the classifier was trained on the training data (`X_train` and `Y_train`), aiming to identify the optimal hyperplane that best separates the classes. Subsequently, predictions were generated for the testing set (`X_test`), and the accuracy metric from `sklearn.metrics` was employed to evaluate model performance. This implementation showcases the practical application of SVM in our analysis, highlighting its efficacy in predicting door status in vertical farming cubes based on environmental parameters.

Figure 15 : Code for Support Vector Machine algorithm

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Initialize the SVM classifier
svm_classifier = SVC()

# Train the SVM classifier on the training data
svm_classifier.fit(X_train, Y_train)

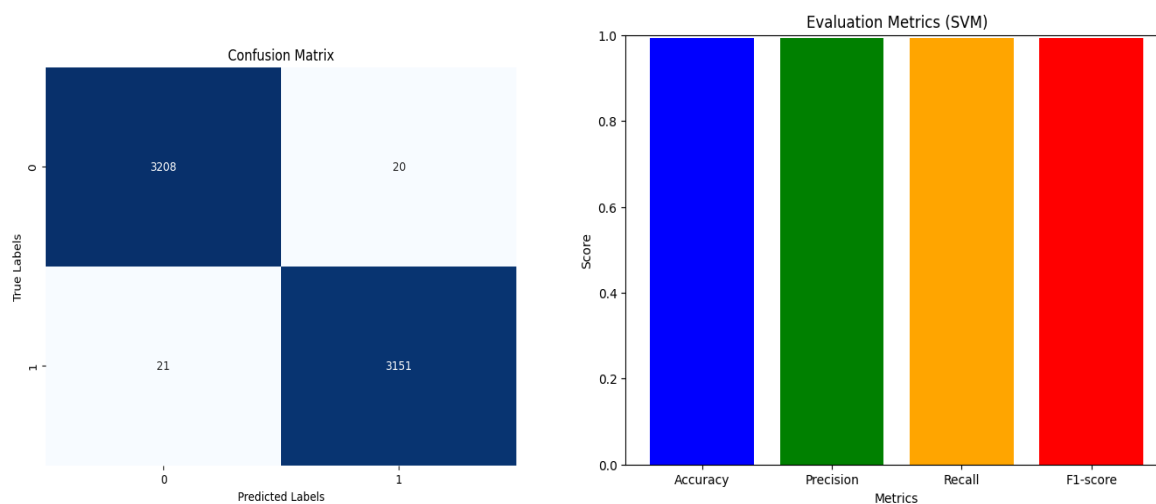
# Predictions on the testing set
Y_pred_svm = svm_classifier.predict(X_test)

# Calculate accuracy
accuracy_svm = accuracy_score(Y_test, Y_pred_svm)
```

8.2.3 Model evaluation

In evaluating the Support Vector Machine (SVM) classifier applied to our dataset, multiple performance metrics were assessed to gauge its effectiveness in predicting door status within vertical farming cubes as shown in [Figure 16](#). The SVM classifier demonstrated exceptional accuracy, achieving a score of 99.36%, indicating its proficiency in accurately classifying instances. Precision, measuring the proportion of correctly predicted positive instances out of all instances predicted as positive, yielded an impressive score of 99.37%, underscoring the classifier's ability to minimize false positives. Similarly, the recall metric, indicating the proportion of correctly predicted positive instances out of all actual positive instances, attained a commendable score of 99.34%, reflecting the classifier's capability to capture true positive cases effectively. The F1-score, representing the harmonic mean of precision and recall, stood at 99.35%, indicating a balanced performance between precision and recall. Collectively, these evaluation metrics affirm the SVM classifier's robustness and accuracy in predicting door status based on environmental parameters, underscoring its potential utility in optimizing operational efficiency and promoting sustainable practices within vertical farming environments.

Figure 16 : Model evaluation graphs



8.3 X_Gradient Boosting

8.3.1 Summary of approach

XGBoost, an acronym for eXtreme Gradient Boosting, was chosen for its exceptional performance in handling structured data and its ability to produce highly accurate predictions. It is renowned for its scalability, speed, and robustness, making it a popular choice for various machine learning tasks.

8.3.2 Applying X_Gradient Boosting

For the implementation of XGBoost on our dataset, we utilized the XGBClassifier module from the xgboost library. Following initialization, the classifier was trained on the training data (X_train and Y_train) using gradient boosting techniques, which sequentially adds models to correct the errors of previous models. Subsequently, predictions were generated for the testing set (X_test), and accuracy was calculated using the accuracy_score metric from sklearn.metrics. This implementation highlights the practical application of XGBoost in our analysis, showcasing its efficacy in predicting door status in vertical farming cubes based on environmental parameters.

Figure 17 : Code for X_Gradient Boosting

```
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score

# Initialize the XGBoost classifier
xgb_classifier = XGBClassifier()

# Train the XGBoost classifier on the training data
xgb_classifier.fit(X_train, Y_train)

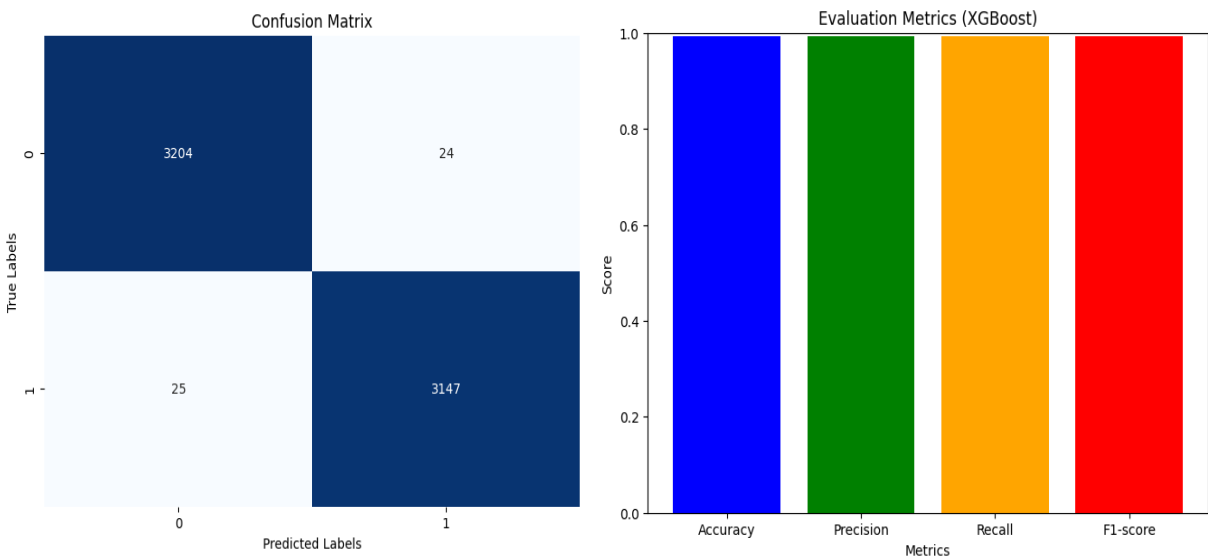
# Predictions on the testing set
Y_pred_xgb = xgb_classifier.predict(X_test)

# Calculate accuracy
accuracy_xgb = accuracy_score(Y_test, Y_pred_xgb)
```

8.3.3 Model evaluation

In evaluating the XGBoost classifier applied to our dataset, a comprehensive assessment of its performance metrics was conducted to ascertain its efficacy in predicting door status within vertical farming cubes as shown in [Figure 18](#). The XGBoost classifier exhibited a remarkable accuracy score of 99.23%, indicating its proficiency in correctly classifying instances. Precision, which measures the proportion of correctly predicted positive instances out of all instances predicted as positive, achieved an impressive score of 99.24%, showcasing the classifier's ability to minimize false positives effectively. Similarly, the recall metric, representing the proportion of correctly predicted positive instances out of all actual positive instances, attained a commendable score of 99.21%, highlighting the classifier's capability to capture true positive cases accurately. Furthermore, the F1-score, serving as the harmonic mean of precision and recall, stood at 99.23%, reflecting a balanced performance between precision and recall. Collectively, these evaluation metrics underscore the robustness and accuracy of the XGBoost classifier in predicting door status based on environmental parameters, reaffirming its potential utility in optimizing operational efficiency and promoting sustainable practices within vertical farming environments.

Figure 18 : Model evaluation graphs



9. Results comparison

Upon comparing the results of the three algorithms - Random Forest, Support Vector Machine (SVM), and XGBoost - applied to our dataset for predicting door status in vertical farming cubes, we observe notable similarities in their performance metrics as shown in [Figure 19](#). All three algorithms achieved exceptionally high accuracy scores, with Random Forest and SVM attaining accuracies of 99.20% and 99.36%, respectively, and XGBoost achieving 99.23%. Similarly, precision, recall, and F1-scores exhibited minimal variations across the algorithms, with each showcasing precision and recall rates above 99% and F1-scores above 99.2%.

While all algorithms demonstrated impressive performance, a nuanced comparison reveals SVM as the top-performing algorithm, achieving marginally higher accuracy and F1-score compared to Random Forest and XGBoost. SVM's ability to handle both linear and non-linear classification tasks effectively, along with its robustness against overfitting, makes it well-suited for our dataset characterized by multiple environmental features and a binary classification task. Additionally, SVM's ability to identify optimal hyperplanes in high-dimensional feature spaces enhances its adaptability to complex datasets, further supporting its selection as the preferred algorithm for predicting door status in vertical farming cubes.

Figure 19 : Comparison Graph

	Model	Accuracy	Precision	Recall	F1-score
0	Random Forest	0.991406	0.990557	0.992119	0.991337
1	SVM	0.993594	0.993693	0.993380	0.993536
2	XGBoost	0.992344	0.992431	0.992119	0.992275

10. Conclusion, Recommendations and Future work

10.1 Conclusion

In conclusion, our analysis demonstrates the efficacy of machine learning algorithms in predicting door status within vertical farming cubes based on environmental parameters. Leveraging Random Forest, Support Vector Machine (SVM), and XGBoost algorithms, we achieved remarkably high accuracy rates, precision, recall, and F1-scores, showcasing their robustness in handling complex datasets and binary classification tasks. SVM emerged as the top-performing algorithm, exhibiting marginally superior performance compared to Random Forest and XGBoost. These findings underscore the potential of machine learning techniques in optimizing operational efficiency and promoting sustainable practices within vertical farming environments.

10.2 Recommendations

Based on our results, we recommend the integration of machine learning-based predictive models, particularly SVM, into vertical farming systems for real-time monitoring and control of environmental conditions. Implementing these models can aid in automating door operations, optimizing resource utilization, and ensuring optimal growing conditions for crops. Additionally, continuous monitoring and evaluation of model performance are essential to maintain accuracy and reliability over time. Collaborative efforts between agricultural experts, data scientists, and technology providers are crucial for the successful implementation and refinement of these predictive models in practical agricultural settings.

10.3 Future Work

Future research endeavors could explore the integration of additional environmental parameters and sensor data to further enhance the predictive capabilities of the models. Additionally, investigating the impact of external factors such as weather conditions and seasonal variations on door status prediction could provide valuable insights for optimizing farming practices. Furthermore, incorporating advanced machine learning techniques such as deep learning and ensemble methods may offer avenues for improving model performance and generalization capabilities. Longitudinal studies monitoring the performance of deployed predictive models in real-world vertical farming environments would provide valuable feedback for iterative refinement and optimization. Overall, continued research and innovation in this domain hold the potential to revolutionize modern agriculture and address pressing global food security challenges.

References

1. Li, J., Lu, X., & Lu, H. (2020). Implementation of machine learning techniques in smart agriculture: A review. *Journal of Integrative Agriculture*, 19(2), 431-446.
2. Singh, V. K., & Misra, A. K. (2020). Application of machine learning in agriculture: A review. *Computers and Electronics in Agriculture*, 173, 105431.

3. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785-794).
4. Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine learning, 20(3), 273-297.